

Müdahale Ederek Anlama: Karmaşık Yazılım Sistemlerini Anlamanın Yöntem Kataloğu

Ömer Faruk Yavuz

Ağustos 2026

Giriş

Problem Alanı

Yazılım mühendisliği literatürü büyük ölçüde inşa etme üzerine kuruludur: tasarım desenleri, mimari stiller, geliştirme metodolojileri, kalite güvence süreçleri. Buna karşılık mevcut ve çalışır durumdaki bir sistemi *anlama* işine ait kavram dağarcığı, hem parçalı hem de kuramsal olarak zayıftır.

Bu asimetri pratikte bir boşluk üretir. Endüstriyel yazılım geliştirmede harcanan zamanın büyük bölümü yeni kod yazmakla değil, mevcut kodu ve mevcut davranışı çözmekle geçer. Bu iş farklı adlar altında dolaşır: hata ayıklama, olay incelemesi, kapasite analizi, kod okuma, mimari arkeoloji, teknik durum tespiti. Adlandırmalar ayrı olduğu için pratikler de ayrı ele alınır; ortak yöntemsel gövde görünmez kalır.

Bu metnin ilk iddiası şudur: bu pratiklerin tamamı tek bir epistemolojik işlemin farklı bağlamlardaki görünümleridir. O işlem, *bilinmeyen bir iç yapının, sınırlı gözlem ve sınırlı müdahale altında çıkarsanması*dır. Bu işlem, bilim felsefesinde çıkarsama, kontrol teorisinde gözlenebilirlik ve tanımlanabilirlik, mühendislikte tersine mühendislik başlıkları altında bağımsız olarak geliştirilmiştir. Yazılım alanı bu birikimi büyük ölçüde ithal etmemiştir.

Merkezî Tez

Metnin merkezî tezi, anlama işinin pasif değil aktif bir işlem olduğudur: *Bir sistemi anlamanın en güçlü yolu onu gözlemek değil, ona kontrollü biçimde müdahale edip tepkisini okumaktır.*

Bu tezin epistemolojik dayanağı, gözlemsel ve müdahaleci bilgi arasındaki farktır. Pasif gözlem birliktelik ilişkisi üretir; bu ilişki, ortak bir üçüncü nedenle veya seçim etkileriyle de açıklanabileceği için nedensel yapı hakkında tekil bir sonuç vermez. Kontrollü müdahale ise karşılığusal bilgi üretir: müdahale edilmeseydi gözlenen sonucun ortaya çıkmayacağı, deneysel olarak gösterilebilir hâle gelir.

Bu ayrım, yazılım alanında son on yılda yaygınlaşan chaos engineering, kanarya dağıtımı, hata enjeksiyonu ve deterministik simülasyon testi gibi pratiklerin ortak kuramsal gerekçesidir. Bu pratikler genellikle araç düzeyinde tartışılır; buradaki amaç, onları ortak bir yöntemsel çerçeveye oturtmaktır.

Kapsam ve Sınır

Metnin kendi ilkesi gereği, kapsamı önce yazılır.

Kapsam içi: çalışır durumdaki bir yazılım sisteminin — kaynak koduna kısmen veya tamamen erişilebilse dahi — davranışsal ve yapısal olarak çıkarsanması; bu çıkarsamanın yöntemleri, sınırları, iktisadi ve kurumsal koşulları.

Kapsam dışı: biçimsel doğrulama kuramının derinliği, güvenlik odaklı ikili analiz teknikleri, makine öğrenmesi sistemlerine özgü yorumlanabilirlik problemleri, donanım-yazılım sınırındaki yan kanal analizi, ve aynı yöntemlerin insan örgütlerine uygulanması. Bunların her biri kendi literatürüne sahiptir ve ayrı ele alınmalıdır.

Metin kuramsal bir katkı iddia etmez. İddiası bütүнleştiricidir: farklı disiplinlerde olgunlaşmış kavramları tek bir çerçevede toplamak, aralarındaki ilişkiyi kurmak ve yazılım pratiğindeki karşılıklarını göstermek.

Yapı

Metin, altı aşamalı bir döngü etrafında düzenlenmiştir: sınır çizme, gözleme, müdahale, çıkarsama, sınır koyma ve aktarım. Bu döngü, iki kuşatıcı koşul içinde işler: gözlemcinin bilişsel ve örgütsel konumu, ve anlamaya ayrılan iktisadi bütçe.

Döngünün dört eksenini ana bölümleri oluşturur. Bunları izleyen bölümler, katalogun kesitsel konularını ele alır: yapılandırılmış arıza analizi; doğrulama, yaşatma ve yenileme süreçleri; gözlemcinin kendisi; iktisat ve meşruiyet; amaç, tarih ve direnç; temsil ve aktarım. Son iki bölüm, çerçevenin iki farklı vaka üzerinde uygulanmasını gösterir.

Ayrı bir bölüm, çerçevenin bilinen sınırlarını ve bu metnin eksiklerini açıkça listeler. Bu bölüm biçimsel bir zorunluluk değil, metnin kendi ilkesinin uygulanmasıdır: belirsizliğin nicemlenmesi ve neyin bilinmediğinin kaydedilmesi, anlama işinin parçasıdır.

Okuyucu

Metin, orta ve ileri düzey yazılım mühendisleri ile mimarları hedefler. Kavramlar tanıtılır ancak temelden öğretilmez; okurun dağıtık sistemler, üretim operasyonu ve yazılım mimarisi konularında çalışma deneyimine sahip olduğu varsayılır. Bu varsayım, kapsamın bir sonucudur: burada tartışılan sınırların çoğu, ancak yeterli sayıda başarısız çıkarsama deneyimlenmiş olduğunda anlamlı hâle gelir.

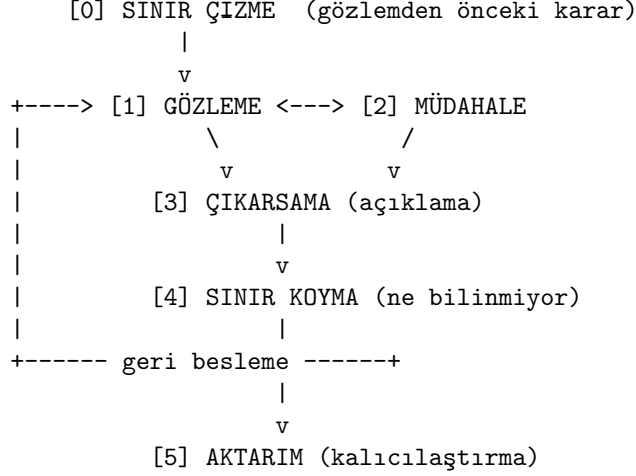
Problem

Karmaşık bir yazılım sistemiyle karşılaşan mühendisin durumu, bir bilim insanının doğa karşısındaki durumuna benzer: sistem oradadır, çalışır, davranış üretir; ama iç kurallarının tamamı hiçbir belgede yazılı değildir. Dokümantasyon eskimiştir, yazarlar ayrılmıştır, mimari kararların gerekçesi kaybolmuştur. Geriye gözlenebilir davranış, kısmi kaynak kodu ve bir yığın örtük varsayım kalır.

Bu durumda yapılan iş, adı konmamış bir tersine mühendisliktir. Bu metnin amacı, o adsız pratiği adlandırmak ve bir yöntem kataloğuna dönüştürmektir.

Merkezî tez: *Bir sistemi anlamanın en güçlü yolu, onu yalnızca gözlemek değil, ona kontrollü biçimde müdahale edip tepkisini okumaktır.* Pasif gözlem korelasyon verir; müdahale karşıolgusal bilgi verir. Çerçevenin adı bu yüzden **müdahale ederek anlamadır.**

Çerçeve: Bir Karar, Dört Eksen, Bir Kapanış



Bunların hepsi şunun içinde çalışır:

[G] GÖZLEMCI | yanlılıklar, organizasyon

[E] EKONOMİ | maliyet, meşruiyet, bütçe

Katalog bir liste değil döngüdür. Sıfıncı adım gözlem değil karardır; beşinci adım anlamının kendisi değil kalıcılaştırılmasıdır. Döngünün tamamı bir gözlemcinin zihninde ve bir kurumun bütçesi içinde çalışır — bu ikisi dışsal koşul değil, sistemin parçasıdır.

Sıfıncı Adım: Sınır Çizme

Sistem Sınırı

Neyi sisteme dahil ettiğin, alacağın cevabı önceden belirler. “Servis yavaş” teşhisi, sınırı sürece çizersen kod optimizasyonuna; veritabanını dahil edersen indeks eksikliğine; ağı dahil edersen DNS çözümleme gecikmesine; istemciyi dahil edersen agresif retry davranışına çıkar. Aynı gözlem, farklı sınırlarla farklı sistemlerin gözlemi hâline gelir.

Pratik kural: bir soruşturmada ilk yazılan şey hipotez değil kapsamdır. “Bu incelemede istemci davranışı ve CDN katmanı dışarıdadır” cümlesi, sonraki bütün çıkarımların geçerlilik alanını tanımlar.

Dışsallıklar

Sınırın dışında bıraktığın her şey ölçümlerinde gürültü olarak geri döner. Yük testinde sanal makinelerin komşu iş yükleriyle kaynak paylaşması (*noisy neighbor*), sınır dışı bırakıldığında açıklanamayan varyans üretir. Sınır kararı, hangi varyansın açıklanacağını hangisinin kabul edileceğini belirler.

Yakın Ayrıştırılabilirlik

Simon'ın gözlemi tüm analizin örtük ön koşuludur: karmaşık sistemler genellikle içeride sıkı, dışarıya gevşek bağlı alt sistemlerden oluşur. Yazılımdaki karşılığı modülerlik, sınırlı bağlam ve arayüz disiplindir. Bu özellik olmasaydı hiçbir parçayı tek başına inceleyemezdik.

Tersi de doğrudur: bir sistemi anlamamanın zorluğu çoğu zaman ayrıştırılabilirliğinin bozulmuş olmasının ölçüsüdür. Global değişebilir durum, gizli zamansal bağlantılar ve paylaşılan veritabanı tabloları ayrıştırılabilirliği yok eder.

Soyutlama Seviyesi ve Kutu Rengi

Sınır çizmenin ikinci boyutu, hangi derinlikte bakılacağıdır. Kara kutu (yalnızca girdi-çıkı), gri kutu (kısmi iç bilgi: metrikler, günlükler), beyaz kutu (kaynak kodu ve iç durum) farklı yöntem setleri gerektirir ve farklı sınırlara çarpar.

Seviye seçimi maliyet-bilgi takasıdır: beyaz kutu daha çok bilgi verir ama daha çok zaman ve erişim ister; kara kutu hızlıdır ama tanımlanabilirlik sınırına daha erken çarpar.

Soyutlama Sızıntısı

Her soyutlama belirli koşullarda altındaki gerçekliği sızdırır. ORM, sorgu planı patolojisi ortaya çıkana kadar veritabanını gizler; TCP, paket kaybı arttığında “güvenilir akış” yanlısamasını kaybeder; dosya sistemi soyutlaması, `fsync` semantığı tartışmaya açılana kadar dayanıklılığı gizler.

Tersine mühendislik çoğu zaman tam olarak bu sızıntı noktalarından başlar: soyutlamanın çöktüğü yer, altındaki mekanizmanın görünür olduğu tek yerdir.

Birinci Eksen: Gözleme

Gözlenebilirlik

Sistemin iç durumunun dış belirtilerden ne kadar okunabildiği. Üç temel kanal farklı sorulara cevap verir: metrik “bir şey bozuldu mu?”, iz “nerede bozuldu?”, günlük “neden bozuldu?”. Yalnızca metrik toplayan bir sistem arızayı fark eder ama yerini bulamaz.

Gözlenebilirlik sonradan eklenen bir özellik değil, tasarım kararıdır. İç durumu dışarı yansıtmayan bir bileşen, kontrol teorisi anlamında gözlenemezdir — ve gözlenemeyen bir sistem güvenilir biçimde kontrol edilemez.

Dağıtık Gözlemin Temel Sınırı

Tek makine sezgisi burada çöker: dağıtık bir sistemde global anlık görüntü yoktur. Duvar saatleri kaymıştır, günlük zaman damgaları sıralama için güvenilmezdir, “aynı anda” ifadesi tanımsızdır.

Bu yüzden olay sıralaması fiziksel değil mantıksal saatle kurulur: Lamport saatleri kısmi sıra verir, vektör saatleri nedensel bağımlılığı yakalar, tutarlı kesit (consistent cut) kavramı “anlık görüntü”nün ne anlama gelebileceğini tanımlar.

Dağıtık izleme sistemlerindeki span ağacı da aynı problemin pratik çözümüdür: bağlam yayılımı ile nedensel zincir taşınır.

Sonuç: dağıtık sistemde “ne olduğunu görmek” bir araç eksikliği değil, teorik bir sınırla kısıtlıdır.

İnformasyon Değeri

Hangi gözlem belirsizliği en çok azaltır? Otuz servisli bir sistemde arıza ararken her sağlık uç noktasını sırayla yoklamak yerine çağrı zincirinin ortasını ölçmek belirsizliği yarıya indirir. `git bisect` aynı ilkedir: ikili arama, gözlem başına maksimum bilgi kazancıdır.

Aktif Öğrenme

Rastgele veri toplamak yerine en öğretici gözlemi seçmek. Fuzzing’in evrimi örnektir: kör rastgele girdi yerine kapsama güdümlü fuzzer’lar (AFL, libFuzzer) yeni kod yolu açan girdileri saklayıp onlardan türetir. Aynı bütçeyle daha çok bilgi çıkar.

Ölçüm Aracının Doğruluğu

Ölçüm zincirinin kendisi bir sistemdir ve o da hata üretir. Toplama aralığı on beş saniye olan bir metrik, üç saniyelik bir doygunluk olayını hiç göstermez. Yüzde birlik örnekleme oranıyla çalışan bir iz toplayıcı, nadir yolu asla yakalamaz. Toplulaştırma sırasında yüzdelik dilimlerin ortalamasını almak matematiksel olarak anlamsızdır ve kuyruk gecikmesini sistematik olarak küçük gösterir.

Kural: bir ölçüm, ölçtüğü olaydan daha hızlı örneklemiyorsa, o olay hakkında kanıt değildir.

Ölçümün Bedeli

Ölçmek sistemi değiştirir. Her fonksiyon çağrısını izleyen bir profil aracı, ölçtüğü şeyi yavaşlatarak sonucu çarpıtır (prob etkisi). Örnekleme tabanlı profil araçları bu yüzden vardır.

Buna Goodhart yasası eşlik eder: bir ölçüt hedefe dönüştüğünde ölçüt olmaktan çıkar. Kod kapsama oranı hedef hâline geldiğinde, hiçbir şey doğrulamayan ama satırları çalıştıran testler yazılır.

Seçilim Yanlılığı

Gözlemleyebildiğin veri kümesi zaten filtrelenmiş olabilir. Wald’ın uçak zırhı problemi yazılımda şöyle görünür: çökme raporu altyapısında biriken kayıtlar, yalnızca raporlamanın ayakta kalabildiği çökmelerdir; süreci anında öldüren hatalar hiç görünmez. Kullanıcı anketleri yalnızca terk etmemiş kullanıcılara ulaşır. Zaman aşımına uğrayan istekler, tamamlanan isteklerin gecikme histogramında yer almaz — yani en yavaş istekler ölçümünden sistematik olarak dışlanır.

Sonuç: kalıntı analizi, sansürlü veri üzerinde yapıldığında yanlış yöne işaret eder.

İkinci Eksen: Müdahale

Kontrol Edilebilirlik

Sistemin hangi durumlarına gerçekten müdahale edebildiğin. Bir hata yolunu tetikleyemiyorsan, o yol hakkında hiçbir şey bilmiyorsun demektir. Bağımlılık enjeksiyonu, saat soyutlaması ve hata enjeksiyon kancaları birer kontrol edilebilirlik yatırımıdır: test edilebilirlik, kontrol edilebilirliğin yazılımdaki adıdır.

Küçük ve Kontrollü Sapma

Sistemi yıkmadan sınırlı bir bozulma uygulayıp tepkiyi okumak. Servis ağı katmanında belirli bir çağrının yüzde birine yapay gecikme eklemek, zaman aşımı yapılandırmasının gerçekten doğru olup olmadığını ortaya çıkarır. Kritik tasarım kavramı *patlama yarıçapı*dır: etkinin önceden sınırlandırılmış olması.

Deney Tasarımı

Müdahale kendiliğinden bilgi vermez; karşılaştırma verir. Kontrol grubu olmadan yapılan bir üretim değişikliği, aynı anda olan her şeyle karışır.

Araçlar: A/B testi (rastgele atamayla kontrol grubu), kanarya dağıtımı (küçük dilime yeni sürüm), gölge trafik (üretim trafiğinin kopyasını yeni servise gönderip cevabı kullanmadan karşılaştırma), faktöriyel tasarım (önbellek boyutu ile iş parçacığı sayısını ayrı ayrı değil kombinasyonlarıyla denemek — etkileşim etkisi tekil etkilerden büyük olabilir), doğal deney (bölgesel kesinti gibi kendiliğinden oluşan ayırmadan yararlanmak).

Müdahale ile Gözlemin Farkı

Bu ayrım katalogun kalbidir. “Yavaş sorgu ile yüksek hata oranı birlikte görülüyor” bir korelasyondur; ikisini de tetikleyen üçüncü bir neden (arka plandaki yedekleme işi) olabilir. Sorguya yapay gecikme enjekte edip hata oranının yükseldiğini görmek ise karşıolgusal bilgi üretir: müdahale etmeseydim bu olmayacaktı.

Chaos engineering’in epistemolojik değeri buradadır. Gözlemsel veriden nedensel çıkarım zordur; müdahaleden doğrudandır.

Üçüncü Eksen: Çıkarsama

Çıkarsama Kipleri

Katalogun geri kalanı bu kiplerin üstünde durur.

- **Tümdengelim:** Kuraldan tekile. “Bu tip asla null olamaz, öyleyse bu dalda null kontrolü ölü koddur.” Tip denetleyicisi ve statik analiz bu kiptedir; kesinliği yüksek, keşif gücü düşüktür.

- **Tümevarım:** Tekilden genele. “Son yüz istekte gecikme 50 ms altında, öyleyse sistem hızlı.” Test paketi bu kiptedir ve Dijkstra’nın uyarısı buraya düşer: test hatanın varlığını gösterir, yokluğunu değil.
- **Abdüksiyon:** Gözlemden en iyi açıklamaya. “Bellek kullanımı testere dişi değil merdiven gibi artıyor; en makul açıklama, bir koleksiyonun büyümesine izin veren referans sızıntısı.” Hata ayıklamanın asıl motoru budur ve en az formalize edilmiş olanıdır.
- **Eleyici çıkarsama:** Hipotezleri düşürerek daralma. `git bisect`, suçlu commit’i eleyerek bulur.
- **Analojik çıkarsama:** Bilinen bir yapıyı bilinmeyene taşımak. Tanımadığın bir kod tabanında “burada muhtemelen bir repository katmanı vardır” beklentisiyle aramak. Güçlü ama tehlikelidir: kalıp benzerliği yapı benzerliği garantisini vermez.

Bayesçi Güncelleme

Her gözlemle hipotez ağırlıklarını değiştirmek. Önsel olarak “yeni deploy edilen kod” hipotezine yüksek ağırlık verirsin, çünkü değişen odur. Günlüklerde deploy öncesine ait aynı hatayı bulduğunda o ağırlık çöker ve altyapı hipotezleri yükselir.

Faydası, sezgisel “en olası şüpheli” önyargısını açık hâle getirmesidir. Sık görülen bir semptomun açıklaması olarak nadir bir mekanizma önermek, önsel olasılığı düşük bir hamledir; önce sıradan mekanizmalar elenmelidir.

Ters Problem

Sonuçlardan nedenlere veya iç parametrelere gitmeye çalışma. Bir API’nin dış davranışından iç veri modelini çıkarmak; ikili dosyanın çağrı desenlerinden protokolünü yeniden inşa etmek; üretim metriklerinden bilinmeyen bir kuyruk yapılandırmasını tahmin etmek. Ters problemler tipik olarak kötü konumlanmıştır: çözüm var olmayabilir, tek olmayabilir veya girdideki küçük gürültüye aşırı duyarlı olabilir.

Kalıntı Analizi

Modelin açıklayamadığı farkın ne söylediğine bakmak. Kapasite modelin trafiğin yüzde doksanını açıklıyorsa asıl bilgi kalan yüzde ondadır. “Ortalama gecikme modelle uyumlu ama 99. yüzdalık dilim üç kat yüksek” gözlemi, ortalamayı değil kuyruğu üreten bir mekanizmaya işaret eder: çöp toplama duraklaması, kilit çekişmesi, bağlantı havuzu tükenmesi.

Mekanizma Açıklaması

“Ne ile birlikte oluyor?” sorusundan “hangi süreç üzerinden oluyor?” sorusuna geçiş. Teşhis ancak adım adım zincir kurulduğunda tamamlanır: bağlantı havuzu

doldu, istekler kuyrukta bekledi, istemci zaman aşımına uğradı, retry gönderdi, yük ikiye katlandı, havuz daha da doldu. Zincir kurulmadan yapılan düzeltme semptom bastırmaktır.

Nedensellik ve Karıştırıcılar

Karıştırıcı değişken, iki gözlemi birbirine bağlıyor gibi görünen üçüncü nedendir. “Yeni sürümü alan kullanıcılarda çökme oranı düşük” gözlemi, yeni sürümü ilk alanların otomatik güncellemeyi açık tutan ve dolayısıyla daha yeni cihazlara sahip kullanıcılar olmasıyla açıklanabilir. Rastgele atama olmadan bu ayrım yapılamaz.

Geri Besleme, Gecikme, Doğrusal Olmama

Yazılım sistemleri düz nedensel zincirler değil, döngüler içerir.

Pozitif geri besleme: yavaşlama retry üretir, retry yükü artırır, yük yavaşlamayı artırır. Negatif geri besleme: otomatik ölçekleme yükü dengeler — ama gecikmelidir, yeni örnekler ısınana kadar sistem hâlâ bozulmaktadır. Gecikmeli negatif geri besleme salınım üretir; ölçek sayısının yukarı aşağı gidip gelmesi bunun gözlenebilir işaretidir.

Doğrusal olmama: yükü iki katına çıkarmak gecikmeyi iki katına çıkarmaz. Doluluk kritik eşiğe yaklaştıkça kuyruk gecikmesi patlar. “Yüzde 50 yükte iyi çalışıyordu” gözlemi, yüzde 80 hakkında neredeyse hiçbir şey söylemez.

Ortaya Çıkış

Parçaların hiçbirinde bulunmayan davranışın bütünde görünmesi. Retry fırtınası, thundering herd, önbellek çığı (aynı anda süresi dolan girdilerin arka uca yığılması), metastabil arıza (tetikleyici ortadan kalktığı hâlde sistemin kendi kendini besleyen bozuk durumda kalması).

Bunların ortak özelliği: her bileşen kendi başına doğru çalışır. Bileşen düzeyinde inceleme bu sınıfı hiçbir zaman bulamaz; yalnızca bütünün dinamiği görünür kılar. Metastabil arıza özellikle öğreticidir, çünkü “nedeni bul ve kaldır” sezgisini kırar: neden zaten kalkmıştır, sistem yine de düzelmez.

Dördüncü Eksen: Sınır Koyma

Tanımlanabilirlik ve Gözlemsel Eşdeğerlik

Eldeki gözlemlerden iç yapının tekil olarak çıkarılıp çıkarılamayacağı. Bir API’nin dışarıdan gözlenen davranışı, arkasında tek bir veritabanı da olsa önbellek destekli üç servis de olsa aynı olabilir. Bu, kara kutu incelemesinin temel sınırır: aynı gözlem kümesini üreten birden çok iç yapı varsa, gözlem onları ayırt edemez.

Pratik sonuç: iç yapıyı gerçekten bilmen gerekiyorsa daha çok ölçmek yetmez, ölçüm *kanalını* değiştirmen gerekir.

Duyarlılık ve Sağlamlık

Hangi parametreye hassas, hangisine dayanıklı olduğunu bulmak. Bir sistemin zaman aşımı değerine aşırı duyarlı olması kırılabilirlik işaretidir: 30 saniye ile 31 saniye arasında davranış niteliksel olarak değişiyorsa orada gizli bir eşik vardır ve bulunmalıdır.

Ölçek Analizi

Aynı kuralın küçük ve büyük ölçekte geçerli olup olmadığını ayırmak. Bin kayıta doğrusal tarama kabul edilebilir, on milyonda değildir. Tek düğümde doğru olan tutarlılık varsayımı üç bölgeye dağıtıldığında geçersizdir. Ölçek değişimi çoğu zaman nicelik değil nitelik değişimidir: yeni bir rejim başlar.

Gereken Çeşitlilik

Ashby'nin yasası: bir sistemi kontrol edebilmek için en az onun kadar çeşitliliğe sahip olmak gerekir. Yazılımdaki karşılığı doğrudandır — sistem on farklı arıza modu üretebiliyorsa ve elde tek müdahale kaldırıcı (yeniden başlatma) varsa, kontrol kapasitesi yapısal olarak yetersizdir. Runbook çeşitliliği, arıza modu çeşitliliğinden az olamaz.

Hesaplamalı İndirgenemezlik

Bazı sistemlerin davranışını öğrenmenin tek yolu onları çalıştırmaktır; kısayol yoktur. Durma probleminin ve statik analizin temel sınırlarının kaynağı budur. Yeterince karmaşık bir eşzamanlılık kurgusunun tüm çalıştırma sıralarını analizle doğrulamak kombinatoriyal olarak patlar; model denetleyicileri bu yüzden durum uzayını sınırlandırarak çalışır.

Pratik sonuç: “daha çok düşünerek” çözülemeyecek problem sınıfları vardır ve bunlar deneyle çözülür.

Kaotik Dinamik ve Öngörü Ufku

Başlangıç koşullarına hassas sistemlerde model doğru olsa bile tahmin belli bir ufkun ötesinde anlamsızdır. Dağıtık sistemlerde iş parçacığı zamanlaması, ağ jitter'ı ve çöp toplama tetikleme anları bu rolü oynar. Yarış koşullarının yeniden üretilmemesi dikkatsizlik değil, sistemin niteliğidir.

Deterministik yeniden oynatma ve kayıtlı hata ayıklama araçları tam olarak bu ufku uzatmak için vardır: rastgelelik kaynaklarını sabitleyerek gözlemi tekrarlanabilir kılarlar.

Belirsizlik Nicemleme

Ne bildiğini ve neyi bilmediğini açıkça ayırmak. Olay sonrası raporlarda “kesin olarak doğrulanan”, “güçlü şekilde desteklenen” ve “varsayılan ama test edilmemiş” ifadelerinin ayrı işaretlenmesi. Bu ayırım yapılmadığında bir varsayım birkaç aktarım sonra olgu olarak dolaşıma girer.

Duhem–Quine Problemi

Hiçbir hipotez tek başına test edilmez; her test arka plan varsayımları demetiyle birlikte sınanır. Test başarısız olduğunda hangi varsayımın yanlış olduğu mantıksal olarak belirsizdir: kod mu hatalı, test mi hatalı, ortam mı farklı, bağımlılık sürümünü mü değiştirdi? Kırmızı bir test, kodun yanlış olduğunu kanıtlamaz.

Model Seçimi ve Aşırı Uyum

Elde birden çok açıklama varsa hangisi tercih edilir? Gözlemleri daha iyi “uydu-ran” model, mutlaka daha doğru model değildir. Yeterince parametre eklenirse her veri kümesi açıklanabilir; sekiz özel durumla donatılmış bir teşhis hipotezi, geçmiş olayların tümünü açıklar ve bir sonrakini öngöremez.

Sadelik burada estetik tercih değil, öngörü gücü kriteridir: iyi model, henüz görülmemiş olayı doğru tahmin edendir.

Arıza Analizi: Yapılandırılmış Yöntemler

İleri ve Geri Yönlü Analiz

Arıza analizinin iki yönü vardır. FMEA (arıza modu ve etkileri analizi) ileri yönlüdür: her bileşen için “bu nasıl bozulabilir ve etkisi ne olur?” sorusu sistematik olarak taranır. Yazılımdaki karşılığı, bir tasarım incelemesinde her dış çağrı için “zaman aşımına uğrarsa, yanlış cevap dönerse, yavaş ama başarılı olursa ne olur?” üçlüsünü sormaktır.

Hata ağacı analizi geri yönlüdür: istenmeyen bir sonuçtan başlayıp onu üretebilecek koşul kombinasyonlarına doğru dallanılır. “Kullanıcı verisi kaybolur” kökünden başlayıp yedekleme, çoğaltma ve silme yollarına inmek gibi.

Kök Neden ve Katmanlı Savunma

Tekil kök neden arayışı çoğu zaman yanıltıcıdır. İsviçre peyniri modeli, ciddi arızaların tek bir hatadan değil, birden çok savunma katmanındaki deliklerin hizalanmasından doğduğunu söyler: kod incelemesi kaçırdı, test kapsamadı, karnarya çok kısa sürdü, alarm eşikliği yanlıştı, runbook eskiydi.

Pratik sonuç: “kök neden” tekil bir kutu değil, bir katmanlar kümesidir ve düzeltme birden çok katmana yapılmalıdır. Yalnızca kodu düzeltip alarmı düzeltmemek, aynı deliği açık bırakmaktır.

Normal Kazalar

Perrow’un tezi: sıkı bağlı ve etkileşimsel olarak karmaşık sistemlerde bazı arızalar öngörülemez ve kaçınılmazdır — kusur değil, yapının doğal sonucudurlar. Buradan iki tasarım yönü çıkar: bağı gevşetmek (asenكرون kuyruk, geri basınç, devre kesici) ve etkileşim karmaşıklığını azaltmak (bağımlılık sayısını düşürmek).

Bu, dayanıklılık felsefesinin “her arızayı önle”den “arızayı sınırla ve toparlan”a kaymasının teorik gerekçesidir.

Doğrulama, Yaşatma, Yenileme

Anlamak yetmez; anlaşılanın güvenilir olduğunun kanıtlanması, zaman içinde yaşatılması ve gerektiğinde yeniden kurulması gerekir. Savunma sanayii mühendisliğinden gelen dört terimin yazılımdaki karşılıkları şunlardır.

Kalifikasyon

“Bu parça gerçekten gereken şartlarda çalışır mı?” sorusunun kanıtlanması. Yazılımda: kabul testleri, uygunluk test paketleri (bir protokol implementasyonunun standarda uygunluğunun sınanması), SLO doğrulaması, hedef yükün iki katında dayanıklılık testi, bağımlılık kesintisi altında davranış kanıtı.

Ayrım önemlidir: fonksiyonel test “doğru mu çalışıyor?” der, kalifikasyon “*beklenen koşul zarfının tamamında* doğru mu çalışıyor?” der. İkincisi zarfın sınırlarının önceden yazılmasını gerektirir.

Benzetim

Gerçek sistemi riske atmadan modelini kurup “şöyle olursa ne olur?” diye denemek. Yazılımdaki biçimleri: kuyruk teorisi modelleriyle kapasite tahmini, deterministik simülasyon testi (tüm eşzamanlılık ve ağ olaylarını sanal bir zamanlayıcı altında çalıştırıp yıllara bedel senaryoyu dakikalarda taramak), gölge trafik, yük modelleri, model denetleme.

Benzetimin epistemolojik değeri, hesaplamalı indirgenemezlik sınırıyla doğrudan ilgilidir: analizle çözülemeyen sistemler, hızlandırılmış çalıştırma ile incelenebilir.

Obsolescence Yönetimi

Bir bileşenin artık üretilmemesi veya desteklenmemesi durumunu önceden yönetmek. Yazılımda: bağımlılık ömür sonu takibi, çalışma zamanı sürüm desteğinin bitmesi, terk edilmiş kütüphaneler, bakımcısı kalmamış paketler, kullanımdan kaldırma politikası ve geçiş takvimi.

Soru şudur: “Bu bağımlılık yarın bakımsız kalırsa sistemi nasıl ayakta tutacağız?” Cevabın önceden verilmemiş olması, teknik borcun en sessiz biçimidir — çünkü hiçbir test kırılmaz, hiçbir alarm çalmaz.

Yeniden Tasarım

Mevcut çözümü aynen kopyalamak yerine işlevini koruyarak yeni bir çözüm üretmek. Yazılımdaki karşılığı, davranışı sabit tutup uygulamayı değiştirmektir: boğucu incir deseni (strangler fig) ile eski sistemi parça parça devralmak, karakterizasyon testleriyle mevcut davranışı önce dondurup sonra iç yapıyı değiştirmek, çift yazma ve karşılaştırma dönemiyle yeni uygulamayı eskisine karşı doğrulamak.

Kritik ayrım: yeniden düzenleme davranışı korur, yeniden yazma korumayı garanti etmez. Yeniden tasarımın ön koşulu, korunacak davranışın *ne olduğunun* önceden çıkarılmış olmasıdır — yani bu metnin tamamı, yeniden tasarımın hazırlık aşamasıdır.

Gözlemcinin Kendisi

Bilişsel Yanlılıklar

Onay yanlılığı hata ayıklamanın en pahalı hatasıdır: bir hipoteze bağlandıktan sonra yalnızca onu destekleyen günlükler okunur. Çapa etkisi, ilk akla gelen açıklamanın sonraki tüm değerlendirmeyi kaydırmasıdır. Uzman körlüğü, sistemi yazan kişinin kendi varsayımlarını görememesidir — bu yüzden yeni katılan birinin naif sorusu çoğu zaman en verimli gözlemdir.

Batık maliyet yanlılığı da buraya düşer: saatlerdir izlenen bir hipotezden vazgeçmek, ona harcanan zaman yüzünden zorlaşır.

Panzehirler

Kırmızı takım (sistemi kırmakla görevli bağımsız grup), premortem (“bu proje altı ay sonra başarısız oldu, nedenini yazın” egzersizi), tahmin kaydı (değişiklikten önce beklenen sonucun yazıya geçirilmesi ve sonra karşılaştırılması), zaman sınırlı hipotez (bir hipoteze en fazla ne kadar süre ayrılacağının önceden kararlaştırılması), suçsuz olay sonrası inceleme.

Sonucusunun gerekçesi ahlaki değil, epistemolojiktir: suçlama varsa bilgi saklanır; bilgi saklanırsa döngü kapanmaz.

Conway Yasası ve Kurumsal Yapı

Bir sistemin yapısı, onu üreten organizasyonun iletişim yapısını yansıtır. Bu doğrudan kullanılabilir bir tersine mühendislik aracıdır: garip görünen bir servis sınırı, teknik gerekçeden çok iki ekip arasındaki eski bir ayrımın izidir. Kodda tıkanıldığında organizasyon şemasına bakmak bazen daha hızlı sonuç verir.

Kurum Düzeyinde Kapasite

Anlama yeteneği bireysel zekâdan çok kurumsal düzenlemenin ürünüdür. Belirleyici olan, başarısızlığı ölçen, kaydeden ve öğrendiğini yeniden üretime sokan mekanizmaların varlığıdır: kod inceleme kültürü, olay sonrası disiplini, iç teknik yazı geleneği, deney altyapısına yapılan yatırım.

Bu, aynı araçlara sahip iki organizasyonun neden çok farklı öğrenme hızlarına sahip olabileceğini açıklar. Araç satın alınabilir; döngüyü kapatan alışkanlık satın alınamaz.

Ekonomi ve Meşruiyet

Bilginin Beklenen Değeri

Her gözlemin maliyeti vardır: mühendis saati, üretim riski, altyapı gideri. Anlama süresiz sürmez. Karar kuralı: bir ölçümün beklenen bilgi kazancı, maliyetinden büyük olmalıdır.

Pratik sonuç: nadir görülen, etkisi düşük bir hatayı tam olarak anlamak rasyonel olmayabilir. “Kök neden bulunamadı, izleme eklendi, tekrarlırsa yeniden bakılacak” meşru bir kapanıştır — belirsizlik açıkça kaydedildiği sürece.

Keşif ve Sömürü Dengesi

Bilinen çözümü uygulamak ile yeni bilgi aramak arasındaki takas. Olay anında denge tamamen sömürü tarafındadır: önce hizmeti geri getir, anlamayı sonraya bırak. Ancak sistematik olarak hep geri alınıp hiç anlaşılmayan bir sistemde arıza oranı zamanla artar, çünkü öğrenme döngüsü hiç kapanmaz.

Etik, Hukuk, Çift Kullanım

Üretimde deney yapmanın meşruiyeti patlama yarıçapının önceden sınırlandırılmış olmasına dayanır: rastgelelik denetimsizlik değildir. Kullanıcı üzerinde yapılan deneylerde (A/B testleri dahil) rıza ve zarar eşiği ayrı bir sorudur.

Hukuki çerçeve yargı bölgesine göre değişir: tersine mühendisliğin birlikte çalışabilirlik ve güvenlik araştırması amacıyla korunduğu alanlar ile sözleşmeyle kısıtlandığı alanlar farklıdır. Üçüncüsü çift kullanımdır: bir sistemi anlamak için geliştirilen araç, onu kırmak için de kullanılabilir.

Amaç, Tarih, Direnç

Tasarım Niyetinin Kurtarılması

Bir yapının biçimi, hangi kısıtlar altında doğduğunu ele verir. “Bu neden böyle yapılmış?” sorusunun cevabı çoğu zaman koda değil koşullara aittir: o dönemki veritabanı sürümü, bir olayın ardından eklenen savunma, artık var olmayan bir istemcinin gereksinimi.

Chesterton’ın çiti buraya düşer: işlevi anlaşılmayan bir kontrolü kaldırmak, onu koyduran koşulun hâlâ geçerli olup olmadığı bilinmeden atılmış bir adımdır. Anlaşılmayan bir if bloğunu silmeden önce, onu ekleyen commit’i ve o dönemin olay kayıtlarını okumak yöntemin parçasıdır.

Yol Bağımlılığı ve Kod Arkeolojisi

Sistemin *neden böyle olduğu* sorusu, *nasıl çalıştığı* sorusundan ayrı bir araç seti ister. Sürüm geçmişi, değişiklik yoğunluğu haritaları, eski mimari kayıtları ve kapatılmış tartışmalar birer katmandır. Katmanları tarihlerine göre okumak, tek bir anlık görüntüde görünmeyen bir yapıyı açığa çıkarır: hangi kararın hangi krize cevap olduğunu.

Histerezis şu anlama gelir: sistem, kendisini oluşturan koşullar ortadan kalktıktan sonra da o koşulların biçimini taşır. Artık kapanmış bir entegrasyon için tasarlanmış veri modeli, tüm yeni geliştirmeyi kısıtlamaya devam eder.

Örtük Bilgi

Bazı bilgiler belgeye aktarılamaz; yalnızca yaparak edinilir. Hangi modülün dokunulmaması gerektiği, hangi testin ara sıra sahte kırıldığı, hangi servisin hangi saatte yavaşladığı yazılı değildir ama takımın belleğindedir.

Bu, bir sistemin kaynak kodunun tamamına sahip olmanın neden onu yeniden üretebilmek anlamına gelmediğinin açıklamasıdır. Aktarım mekanizması dokümantasyon değil, birlikte çalışmadır: eşli programlama, çıraklık, gölge nöbet dönemi.

Direnç: Anlaşılmaya Karşı Tasarım

Anlamaya çalıştığınız sistem seni engelliyorsa metodoloji değişir. Kod karartma, kurcalama koruması, hata ayıklayıcı tespiti, minifikasyon, belgelenmemiş protokoller. Gözlem kanalları daralır, müdahale maliyeti artar; yöntem dolaylı kanallara kayar — zamanlama, bellek deseni, ağ trafiği.

Madalyonun diğer yüzü savunmadır: kendi sisteminde neyi gözlenebilir bıraktığınız, saldırganın da ne öğrenebileceğini belirler. Ayrıntılı hata mesajı, hata ayıklama kolaylığı ile bilgi sızıntısı arasındaki takas noktasıdır.

Temsil ve Aktarım

Notasyon Bir Anlama Aracıdır

Aynı sistem farklı temsillerde farklı şeyleri görünür kılar. Bir ödeme akışını sıralı diyagram olarak çizmek zamansal sırayı, durum makinesi olarak çizmek geçersiz geçişleri, veri akış diyagramı olarak çizmek güven sınırlarını görünür kılar. Yanlış notasyon seçimi, mevcut olan bilgiyi görünmez yapar.

Buna ontoloji çalışması eşlik eder: kavramları adlandırmak. Ortak dil (ubiquitous language) bir iletişim aracı olmadan önce bir anlama aracıdır; adı olmayan ayırım üzerinde düşünülemez.

Metamodeller

Sistemin kendisinden çok, onu açıklamak için kurulan modelleri kıyaslamak. Aynı gecikme problemi kuyruk teorisi modeliyle, kaynak çekişmesi modeliyle ve bağımlılık grafiği modeliyle açıklanabilir. Hangisinin daha iyi olduğu, doğruluktan çok hangi müdahaleyi mümkün kıldığına bakılarak belirlenir.

Anlamayı Kalıcılaştırma

Anlaşılan şey kaydedilmezse döngü kapanmaz; bilgi kişide kalır, kurumda kalmaz.

- **Regresyon testi:** Bir kez anlaşılmış arızayı koşulabilir bilgiye çevirmek. Her düzeltmenin yanında bırakılan test, o hatanın öğrenilmiş olduğunun kanıtıdır.
- **Karakterizasyon testi:** Doğru olduğu bilinmeyen ama mevcut olan davranışı dondurmak. Yeniden tasarımın ön koşulu.

- **Mimari karar kaydı:** Kararı değil gerekçesini ve reddedilen alternatifleri saklamak. Tasarım niyetinin gelecekte kurtarılmasını gereksiz kılan tek yöntem.
- **Runbook:** Teşhis prosedürünün adım adım yazımı; örtük bilginin kısmen açık hâle getirilmesi.
- **Koşabilir spesifikasyon:** Belgenin eskimesini engellemek için doğruluğunun otomatik sınanabilir olması.
- **Olay sonrası inceleme:** Tek olaydan çıkan bilginin tasarıma, alarmlara ve prosedürlere geri taşınması.

Vaka İncelemesi: Chaos Engineering

Araç ve Tez

Chaos Monkey, çalışan bir bulut sisteminde rastgele seçilmiş tekil bir sunucu veya uygulama örneğini devre dışı bırakır ve tek bir soruyu sorar: sistem, kullanıcı fark etmeden çalışmaya devam ediyor mu? Amaç zarar vermek değil, zayıflığı erken bulmaktır.

Arkasındaki tez: sunucuların bozulması istisna değil kaçınılmazdır; sistemi buna rağmen çalışacak şekilde tasarla. Operasyonel karşılıkları: tek makineye güvenme, arızada insan müdahalesini bekleme, hata toleransını varsayma ve düzenli kanıtlarla, hataları kullanıcı bulmadan sen bul.

Katalogdaki Karşılıkları

- **Sınır çizme:** Kapsam, sıklık ve istisna listeleri; patlama yarıçapının önceden tanımlanması.
- **Müdahale:** Küçük ve kontrollü sapma; nedensel bilginin gözlemden değil müdahaleden çıkarılması.
- **Gözleme:** Bozulma anında metrik, iz ve alarm üzerinden iç durumun okunması.
- **Çıkarsama:** “Bu hizmet bir sunucu kaybına dayanır” hipotezinin sınanması ve gerekirse çürütülmesi; bağımlılık zincirinin açığa çıkması.
- **Sınır koyma:** Hangi arıza modlarının test edildiğinin, hangilerinin edilmediğinin kaydedilmesi.
- **Kalifikasyon:** Dayanıklılığın varsayım değil, tekrarlanan kanıt hâline gelmesi.
- **Aktarım:** Bulgunun tasarıma, alarm eşiklerine ve runbook'lara geri taşınması.

Yan Ürün

Netflix'in birincil amacı sürekli dayanıklılık doğrulamasıdır. Ancak yan etkisi bu metnin ana temasıyla örtüşür: görünmeyen bağımlılıkları ve sessiz varsayımları görünür ve adlandırılabilir hâle getirir. Yazılı olmayan bilgiyi yazılabilir hâle çevirir.

Vaka İncelemesi: Yabancı Bir Kod Tabanına Giriş

Katalog, tek bir yaygın senaryoda sırayla uygulanabilir.

1. **Sınır çizme:** Hangi servisler kapsamda, hangileri kara kutu. Kapsam yazılır. Hangi soyutlama seviyesinde çalışılacağına karar verilir.
2. **Yapısal gözlem:** Dizin yapısı, bağımlılık grafiği, derleme yapılandırması. Ayrıştırılabilirliğin nerede korunduğu, nerede bozulduğu.
3. **Tarihsel gözlem:** En çok değişen dosyalar, en çok yazarı olan modüller, en uzun süredir dokunulmamış bölgeler. Sırasıyla risk, karmaşıklık ve fosil alanları.
4. **Dinamik gözlem:** Tek bir isteği uçtan uca izleme. Statik okumanın asla veremeyeceği bilgi budur: hangi yolun gerçekten çalıştığı.
5. **Abdüktif hipotez:** Gözlenen yapıyı üretebilecek en makul tasarım niyeti nedir.
6. **Tahmin kaydı:** Müdahaleden önce beklenen etki yazılır.
7. **Müdahale:** Küçük ve geri alınabilir bir değişiklik yapılır, sonuç tahminle karşılaştırılır.
8. **Kalıntı:** Beklenen ile gözlenen arasındaki fark. Modelin yanlış olduğu yer buradadır ve en çok bilgi buradadır.
9. **Sınır koyma:** Neyin doğrulandığı, neyin hâlâ varsayım olduğu açık ayrımla kaydedilir.
10. **Aktarım:** Öğrenilen her şey karakterizasyon testiyle, mimari karar kaydıyla veya runbook'la kalıcılaştırılır.

Kapanış

Katalogun maddeleri tek bir disiplinin alt başlıkları değildir; mühendislik, bilim felsefesi, kontrol teorisi, tasarım ve örgütsel öğrenmenin kesişiminde duran bir araç ailesidir. Onları birbirine bağlayan şey konu değil kiptir: hepsi *bilinmeyen bir yapıyı, sınırlı gözlem ve sınırlı müdahaleyle çıkarsama* işini yapar.

Dört ilke katalogun tamamını özetler.

- **Nedensel bilgi müdahaleden gelir.** Pasif gözlem korelasyon üretir; kontrollü sapma karşıolgusal bilgi üretir.

- **Anlamanın sınırları bilinmeden anlama tamamlanmaz.** Tanımlanabilirlik, indirgenemezlik, öngörü ufku ve seçim yanlılığı, ne kadar ilerlenebileceğinin haritasını çizer.
- **Gözlemci sistemin parçasıdır.** Yanlılıklar, organizasyon yapısı ve bütçe, ne göreceğini gözlemden önce belirler.
- **Kaydedilmeyen anlama, anlama değildir.** Kişide kalan bilgi kurumda kalmaz; döngü ancak aktarımla kapanır.

Bir sistemi anlamak, sonunda onun hakkında doğru cümleler kurabilmek değildir. Ona doğru müdahaleleri yapabilmek, hangi müdahalelerin işe yarayacağını önceden bilebilmek ve bu bilgiyi kendinden sonrakine aktarabilmektir.

Çerçevenin Sınırları ve Bu Metnin Eksikleri

Katalog, kendi ilkesine tabidir: neyin dışarıda bırakıldığı ve nerede zayıf kaldığı açıkça kaydedilmelidir. Aksi hâlde metin, uyardığı hatayı kendisi işler — kapsamı belirtilmemiş bir çıkarsama, geçerlilik alanı tanımsız bir sonuç üretir.

Yöntemsel Sınırlar

Ampirik temel yoktur. Metin analitiktir; sunulan çerçevenin uygulandığı ve uygulanmadığı takımlar arasında ölçülmüş bir fark gösterilmemiştir. Kavramların pratik değeri, burada argümanla savunulmuştur, veriyle değil.

Öncelik sıralaması verilmemiştir. Katalogdaki kavramlar biçimsel olarak eşit ağırlıkta sunulur. Oysa pratikte küçük bir alt küme — gözlenebilirlik, mekanizma açıklaması, kontrollü sapma, kalıntı analizi — vakaların büyük çoğunluğunu karşılar. Hangi kavramın hangi sıklıkta işe yaradığına dair bir ağırlıklandırma yapılmamıştır.

Karşı durum işlenmemiştir. Çerçevenin ne zaman fazla geldiği yeterince tartışılmamıştır. Basit, düşük etkili veya iyi anlaşılmış problemlerde sistematik çıkarsama yürütmek kaynak israfıdır; doğru cevap çoğu zaman bilinen düzeltmeyi uygulayıp devam etmektir. İktisat bölümü bu soruna değinir ancak karar eşiğini işlemez.

Yöntemin kendisi doğrulanmamıştır. Metin, bir sistemi anlamanın yöntemini önerir ama bu yöntemin doğruluğunu kendi ölçütleriyle sınamaz. Çerçeve, kendi tanımlanabilirlik sorusuna tabidir: aynı pratik başarıyı üretebilecek başka çerçeveler var mıdır ve bu metin onları ayırt edebilir mi? Cevap verilmemiştir.

İçerik Eksikleri

Biçimsel yöntemler yüzeysel kalmıştır. Model denetleme, zamansal mantık, tip teorisi ve tasarımıyla kanıt yaklaşımları yalnızca hesaplamalı indirgenemezlik bağlamında anılmıştır. Bunlar, gözlemsel çıkarsamaya alternatif bir bilgi kaynağı sunar ve ayrı bir eksen oluşturmaya hak eder.

Güvenlik boyutu dar tutulmuştur. Tehdit modelleme, saldırı ağaçları, ikili analiz ve güven sınırı çıkarımı yalnızca “direnc” altbaşlığında ele alınmıştır. Düşmanca bir bağlamda çıkarsama, işbirlikçi bağlamdakinden yöntemsel olarak farklıdır; bu fark işlenmemiştir.

Makine öğrenmesi sistemleri kapsanmamıştır. Bu sistemlerde iç yapı ile davranış arasındaki ilişki, klasik yazılımdakinden niteliksel olarak farklıdır: yorumlanabilirlik, veri kayması, dağılım dışı davranış ve model çıkarma saldırıları ayrı bir kavram kümesi gerektirir. Katalog buraya kısmen uygulanabilir ancak yeterli değildir.

Veri katmanı zayıf işlenmiştir. Şema evrimi, veri kalitesi patolojileri, geçmiş göçlerin bıraktığı tutarsızlıklar ve üretim verisinden iş kurallarının çıkarsanması yalnızca dolaylı olarak geçer. Pratikte bir sistemin en dayanıklı ve en az belgelenmiş bilgisi çoğu zaman veri modelinde saklıdır.

Görsel temsil kullanılmamıştır. Temsil bölümü notasyon seçiminin bir anlama aracı olduğunu savunur ancak metnin kendisi neredeyse tamamen düzyazıdır. Geri besleme döngüleri, katmanlı savunma ve hata ağaçları doğaları gereği diyagramla daha iyi aktarılır. Bu, metnin kendi tezine karşı bir tutarsızlığıdır.

Biçimsel Sınırlar

Örnekler illüstratiftir, vaka değildir. Yazılım örnekleri kavramları göstermek için kısa tutulmuştur; baştan sona işlenmiş, gerçek verilerle desteklenmiş tek bir soruşturma anlatısı yoktur. Bu, kavramların birbirine nasıl bağlandığını göstermeyi zorlaştırır.

Kaynakça verilmemiştir. Simon, Ashby, Perrow, Quine, Conway ve diğerlerine yapılan atıflar kavramsaldir ve birincil kaynaklara bağlanmamıştır. Metin bu hâliyle bir başvuru düzenlemesidir, bir literatür incelemesi değildir.

Metin katalog ile tez arasında gerilimlidir. Giriş bölümü tek bir sav öne sürer; gövde ise ansiklopedik bir düzenlemeye yönelir. Sonuç olarak metin baştan sona okunmaktan çok başvuru amaçlı kullanılmaya uygundur. Bu bir kusur olarak değil, bir tür seçimi olarak kaydedilmelidir — ancak seçim açıkça yapılmamıştır.